

I/O Devices

- CPU communicates with I/O devices using Data, Status and Configuration registers
- Device mechanism may be digital, analog, optical, mechanical, etc.
- Program running on CPU uses registers to configure device, initiate a device action, detect device state, determine action completion, etc.

Synchronous vs Asynchronous

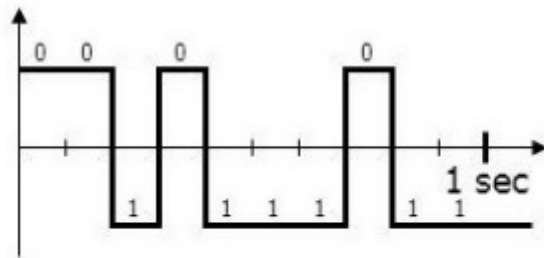
- A **synchronous** serial communication is always coupled with a CLOCK signal - I2C and SPI
- An **asynchronous** serial communication, on the other hand, does not use an external CLOCK signal and therefore requires less I / O channels for communication. The most common serial protocols are Bluetooth, Xbee, GPS module.

Serial communication parameters

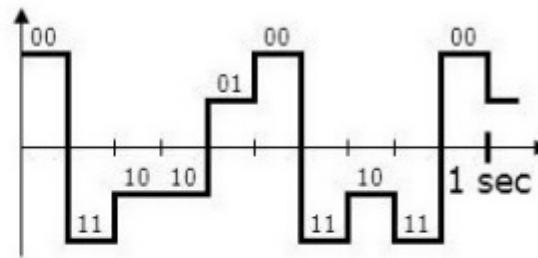
- Baud (bit) rate (determines pulse width)
- Number of bits per character (6, 7, or 8)
- Parity/no parity
- Even/odd parity
- Length of stop bit (1, 2 bits)

Baud rate vs Bit rate

- Baud Rate: How many times a signal changes per second
- Bit Rate: How many bits can be sent per time unit (usually per second)
- Bit Rate: is controlled by baud and number of signal levels.

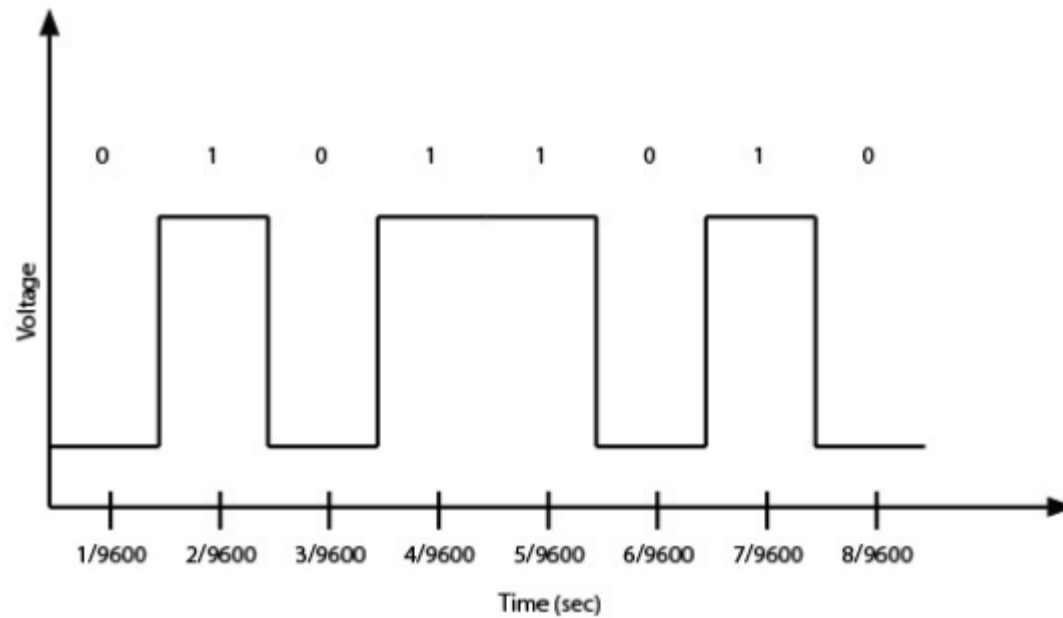


Baud = 10
Bit rate = 10 bps



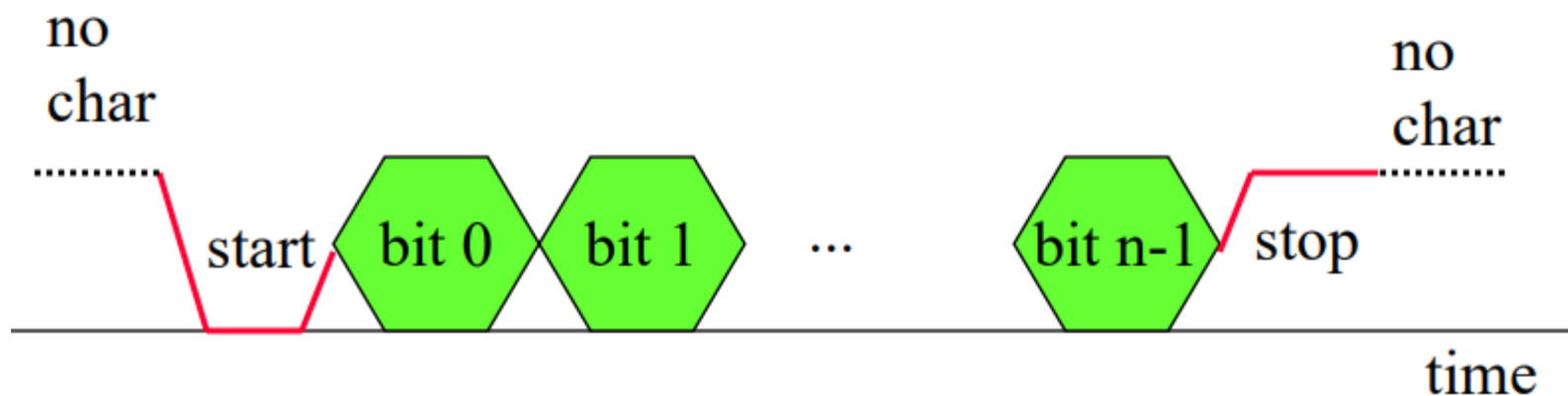
Baud = 10
Bit rate = 20 bps

Example: Serial Data Transmission of 8 bits

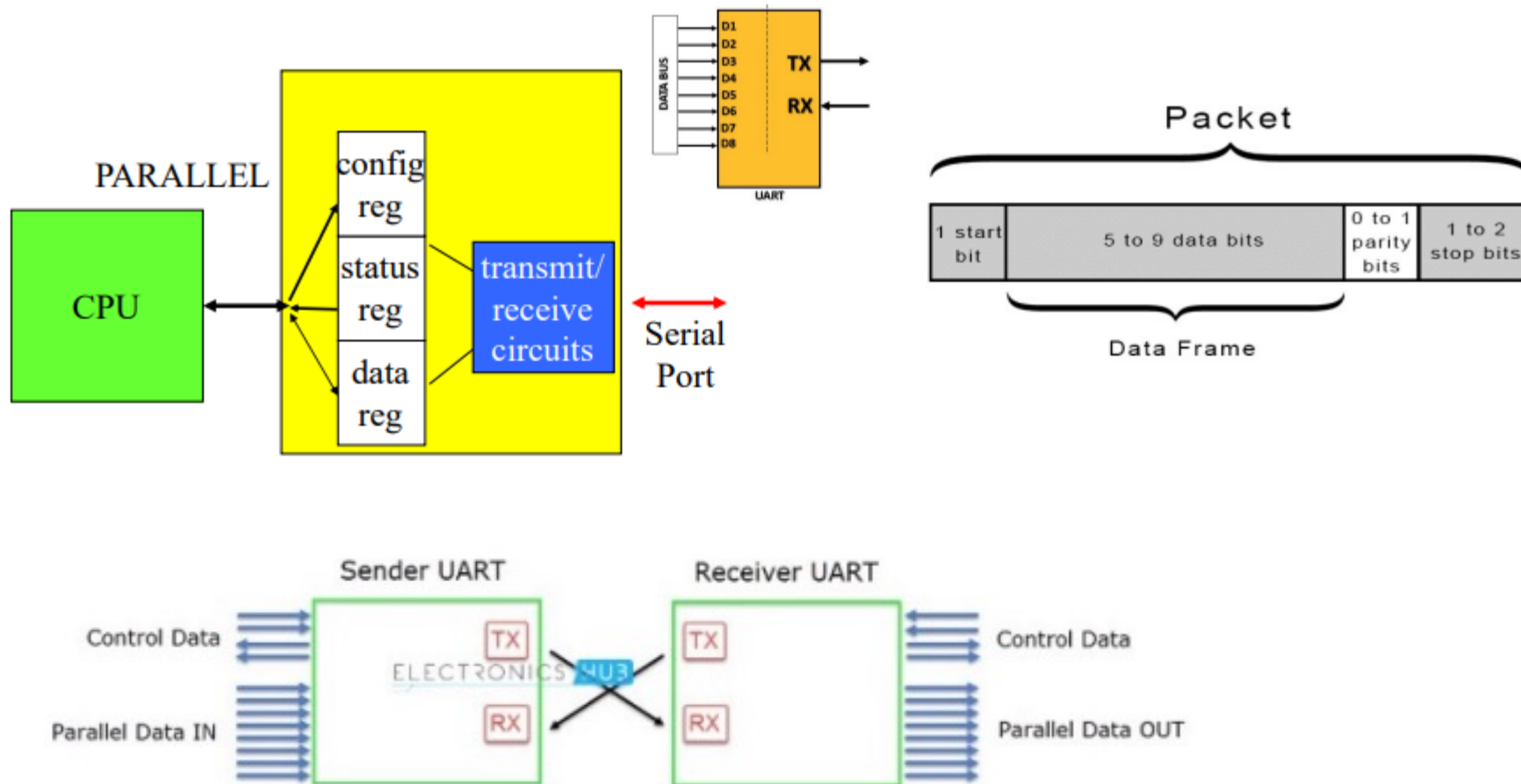


Universal Asynchronous Receiver Transmitter (UART)

- Allow you use PC applications to communicate with the board using standard Windows COM port commands.
- Bytes are transmitted serially using a sequence of equally spaced high/low pulses that encode bits
- Bracketed by Start, Stop pulses
- Sender/receiver assume the same rate, use local clocks

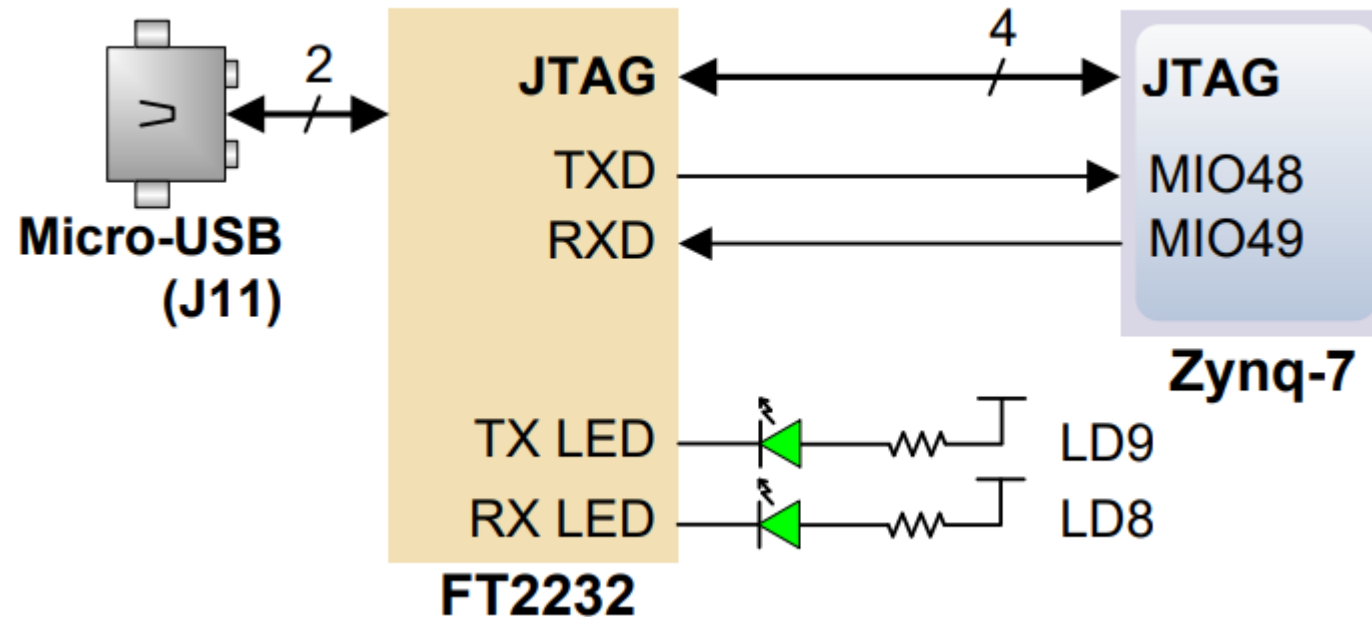


UART Device



UART Device

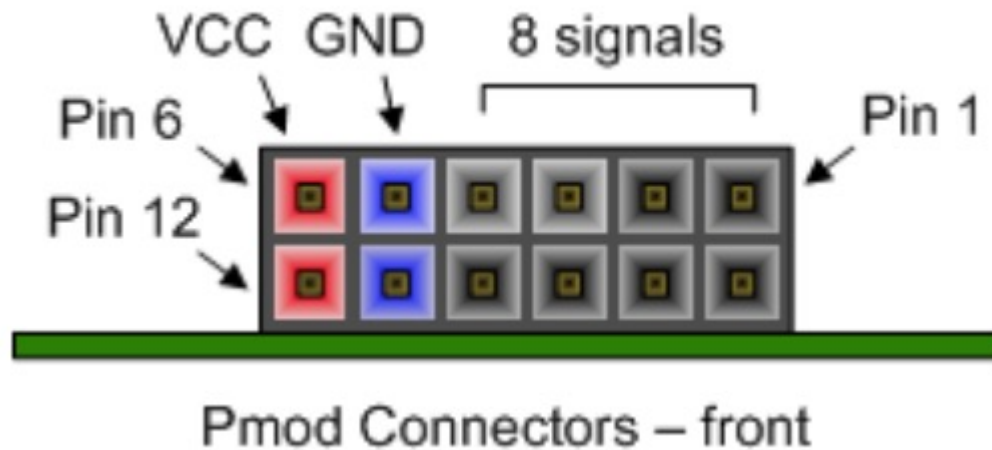
- Default protocol parameters: 115200 baud rate, 1 stop bit, no parity, 8-bit character length.



- Example: https://github.com/mu-aslam/Zynq_7000_SDK/blob/UART_MIO/project_1.sdk/test_uart/src/helloworld.c

UART Device for PL

- A Digilent PmodUSBUART needed
- The USB-UART bridge is the FTDI FT232RQ device (www.ftdichip.com).



- Serial port data is exchanged with the Zynq PL using a two-wire serial port (TXD/RXD).

UART Device for PL

- rcvr and txmit Verilog

```
module rcvr (output reg [7:0] rbr, output reg rdrdy, input rxd, clk16x, rdrst)
```

rbr	8-bit receive data output
rdrdy	receive data ready
rxd	serial data input
clk16x	clock at 16 times bit rate
rdrst	receive data reset

```
module txmit (input [7:0] tdin, output reg tbre, input txrst, clk16x, wrn, output reg txd);
```

tdin	8-bit transmit data input
tbre	transmit buffer empty
txrst	transmit data reset
clk16x	clock at 16 times bit rate
wrn	write transmit data
txd	serial data output

UART Advantages and Disadvantages

- Advantages of UART

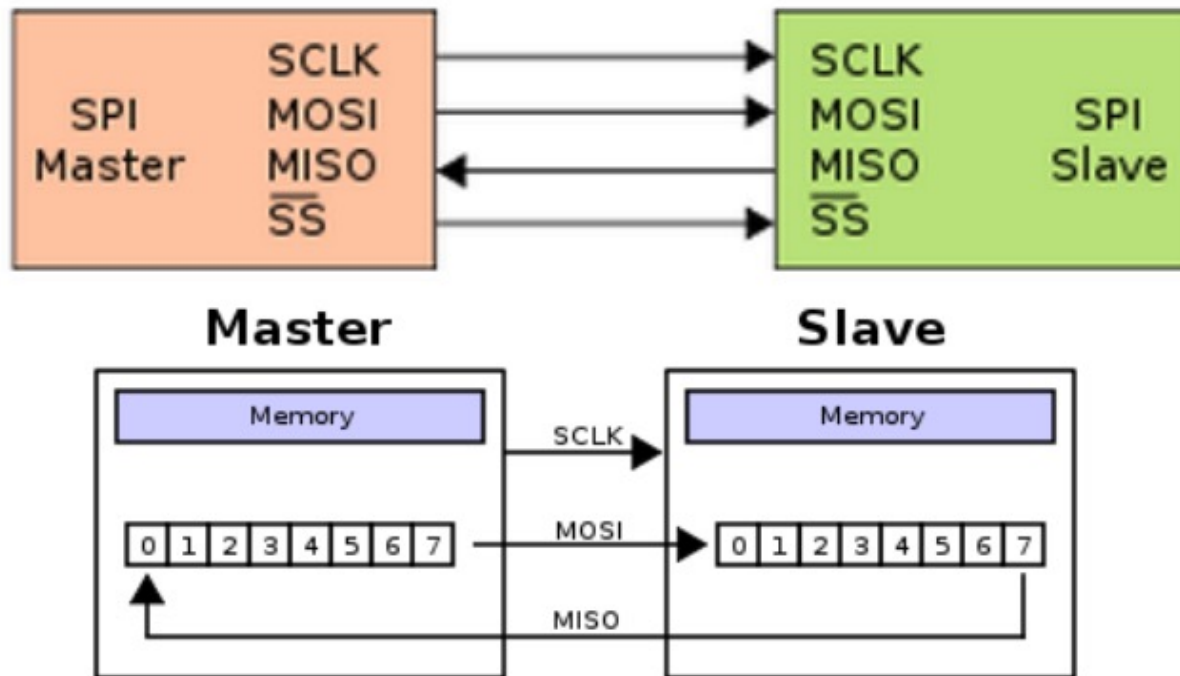
- Requires only two wires for full duplex data transmission (apart from the power lines).
- No need for clock or any other timing signal.
- Parity bit ensures basic error checking is integrated in to the data packet frame.

- Disadvantages of UART

- Size of the data in the frame is limited.
- Speed for data transfer is less compared to parallel communication.
- Transmitter and receiver must agree to the rules of transmission and appropriate baud rate must be selected

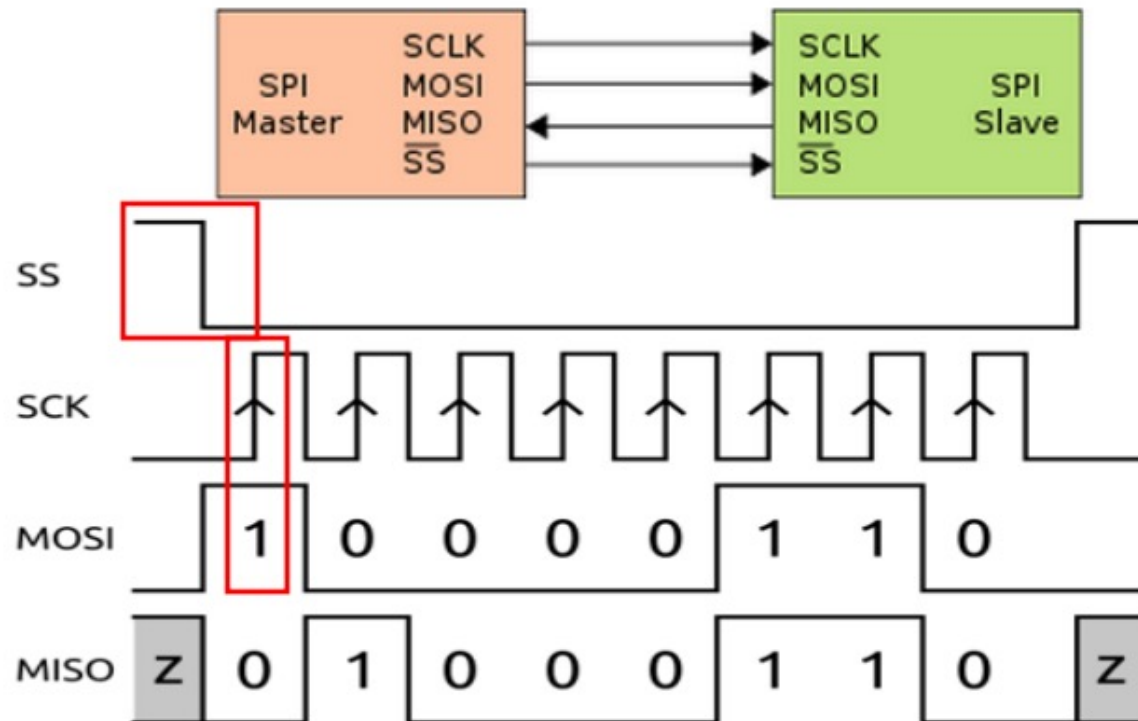
Serial Peripheral Interface (SPI)

- The Serial Peripheral Interface (SPI) consists of the serial clock SCLK, master output/slave input MOSI, master input/slave output MISO and slave select $\overline{SS}$ signals.



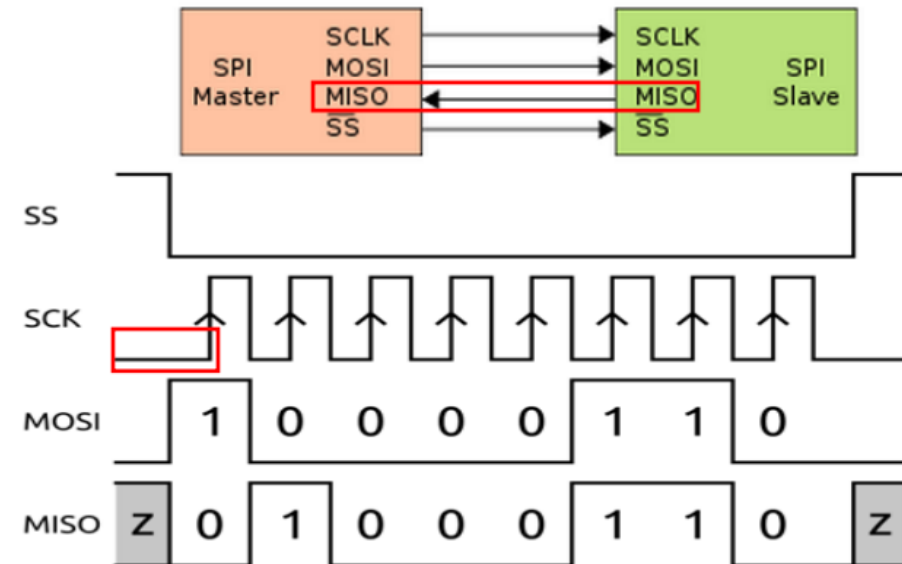
Serial Peripheral Interface (SPI)

- Data transmission begins with the SS signal at logic 0.
- Either the rising (as shown) or falling edge of SCLK as specified by the slave device signifies valid MOSI data with the most significant bit (MSB) sent first.



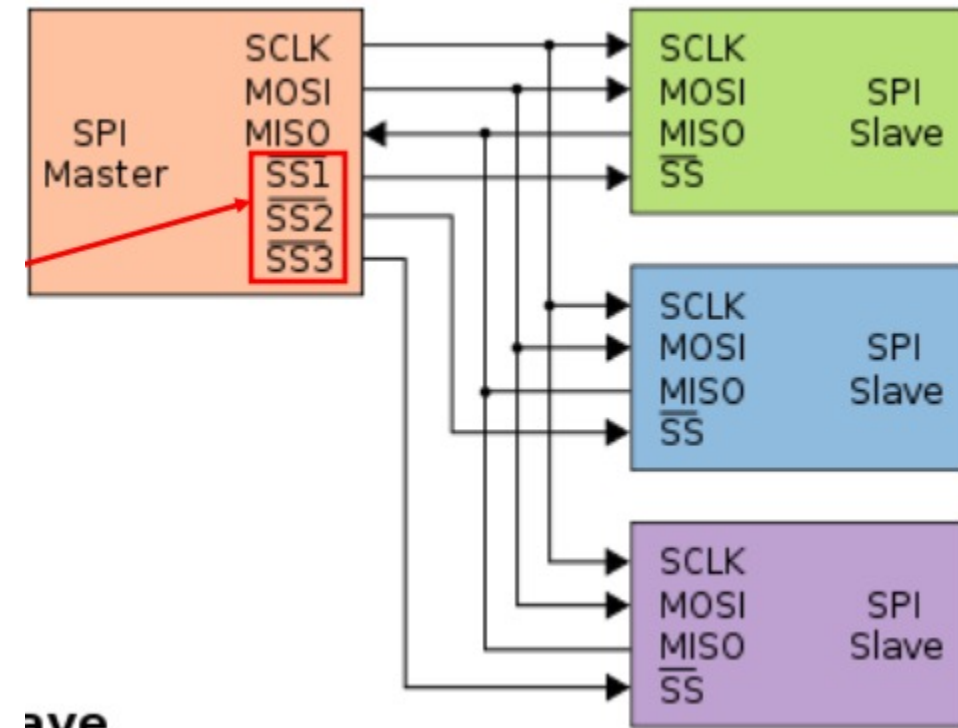
Serial Peripheral Interface (SPI)

- The default value of the SCLK signal for a rising edge valid data transmission is logic 0 and for a falling edge it is logic 1.
- Concurrently, the MISO signal outputs the MSB of the last (not the current) data transmission to the SPI bus master device.
- This serial data transmission continues with any number of SCLK cycles until the SS signal returns to logic 1.



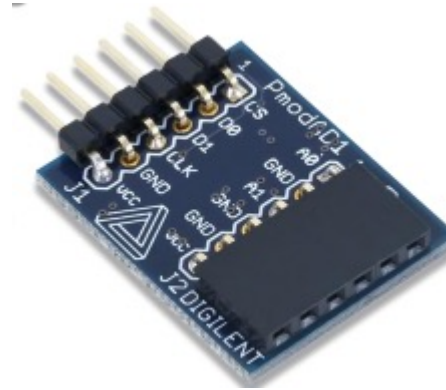
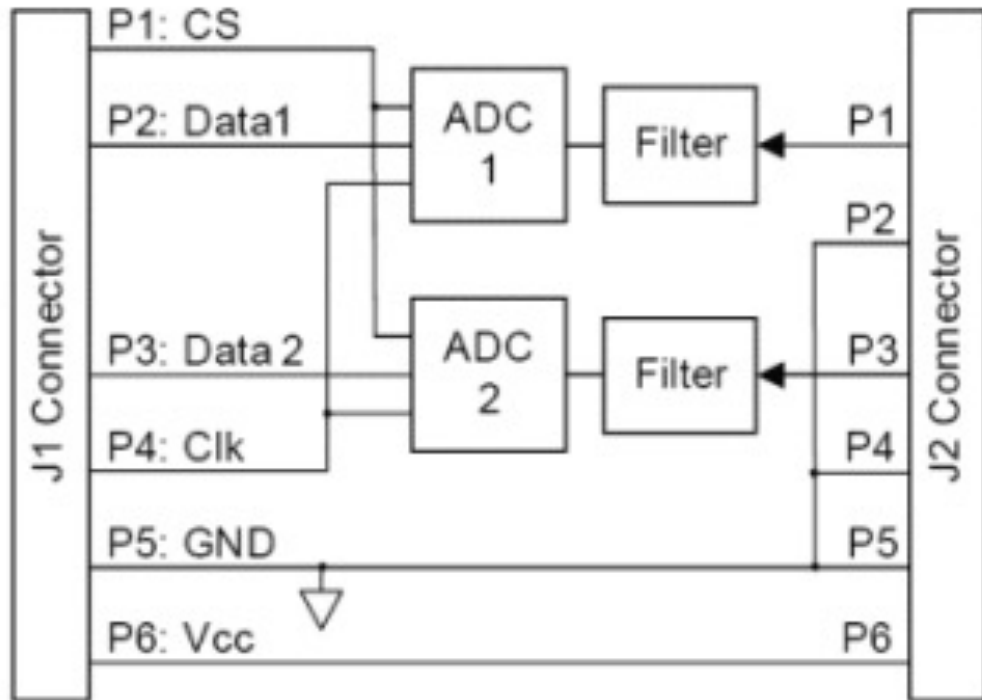
SPI Bus

- Multiple independent SPI slave devices can be interfaced to a single SPI master with multiple SS signals.
- Alternatively, Multiple cooperative SPI slave devices can be interfaced to a single SPI master with a single SS signal and daisy chaining the MISO and MOSI signals



SPI Example: ADC

- A Digilent Pmod AD1 two-channel, 12-bit resolution analog-to-digital converter (ADC) is interfaced to the Zynq PS in the Vivado HLx design project using the extended ZynqPmodVivado-Library.



SPI Example: ADC

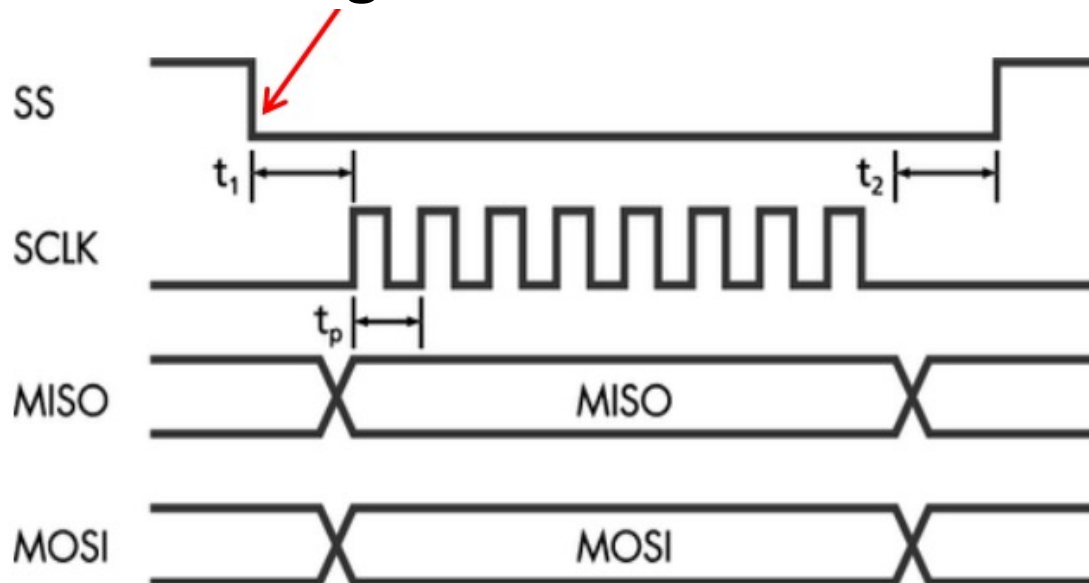
- The general equation for an n-bit binary data output $D[n-1:0]$ for the ADC that inputs an analog signal is:

$$D[n-1:0] = G \frac{V_{IN} - V_{REF}}{V_{FS}} 2^{n-1}$$

- The maximum rate at which the ADC can sample an analog signal and produce n-bit binary data is its throughput rate R_{ADC} in samples per second.

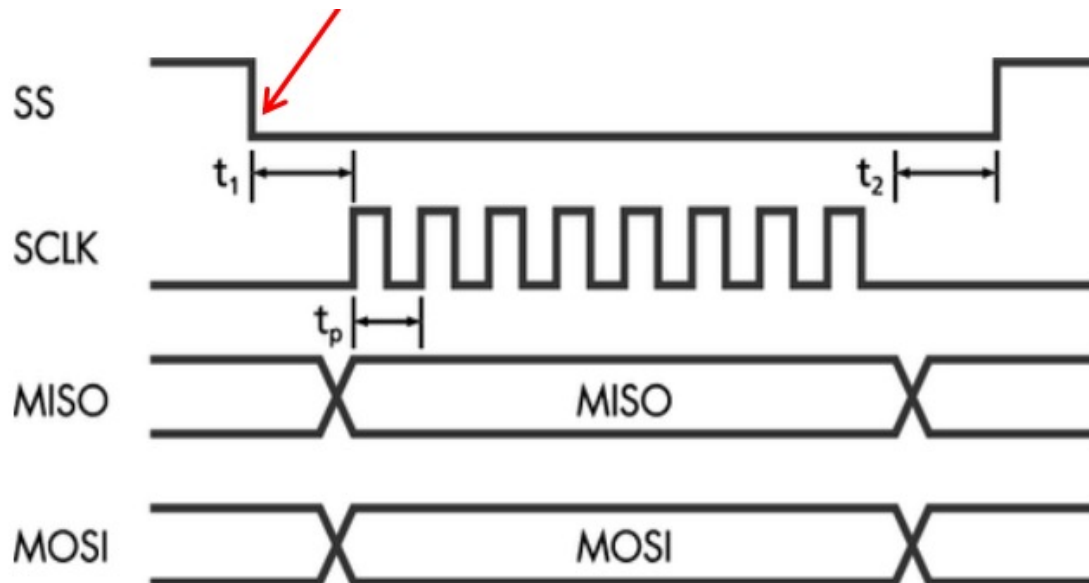
SPI Example: ADC

- The Pmod AD1 ADC uses $G = 1$, $V_{REF} = 0$, $V_{FS} = V_{DD} = 3.3\text{ V}$ and $n = 12$ for a digital range of 0 to 4095 for an input voltage for 0 to 3.3 V.
- The ADCs use an active low ADCCS signal and an SPI bus clock signal SCLK and output two data signals ADCD0 and ADCD1. The ADC SPI data communication protocol begins with the chip select signal ADCCS set to logic 0 which initiates analog data conversion.



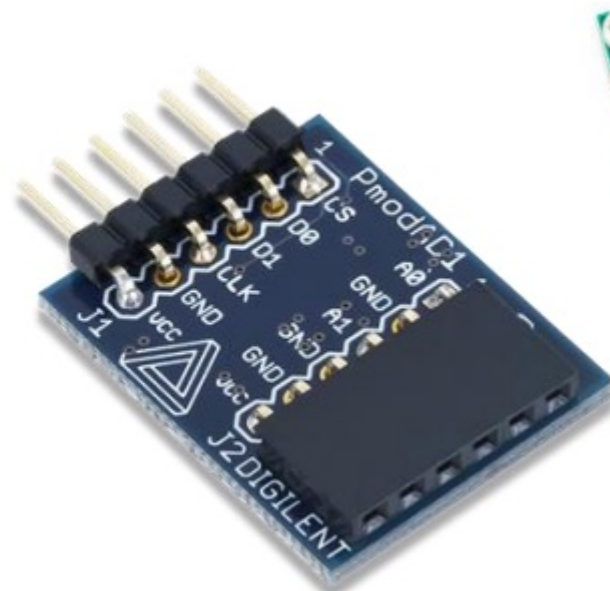
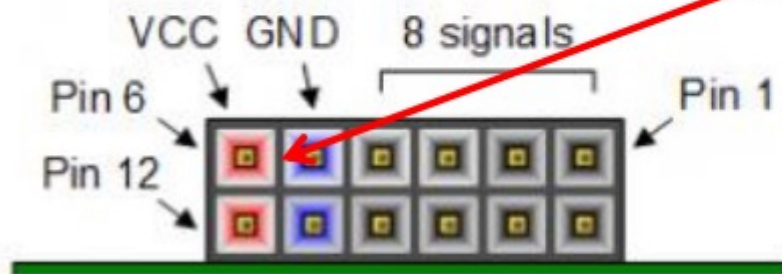
SPI Example: ADC

- The Pmod AD1 ADCD7476 ADC uses the falling edge of the SPI clock signal SCLK to transmit a 16-bit data packet, with the first four bits as don't care bits and the most significant bit (MSB) of the data sent first. After all 16 bits have been sent, the chip select signal ADCCS is set to logic 1.



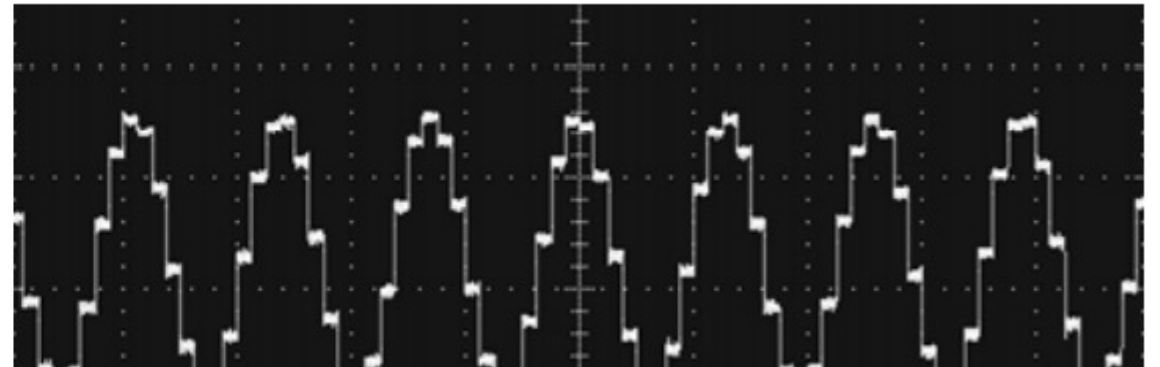
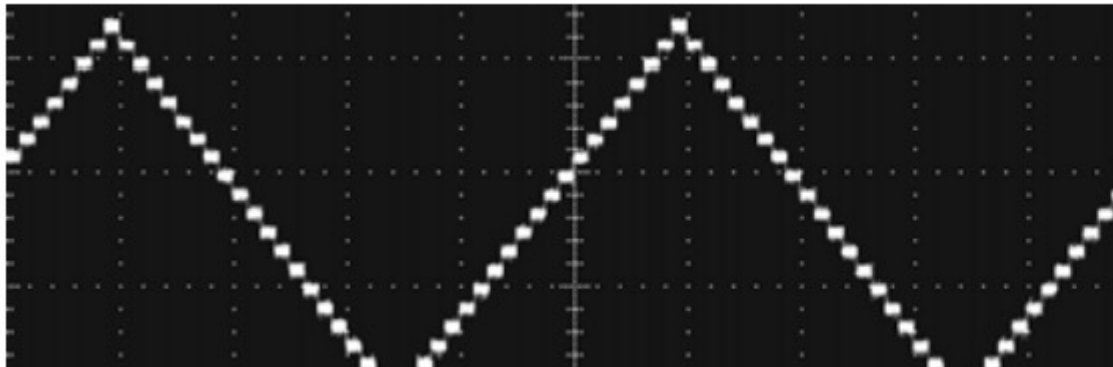
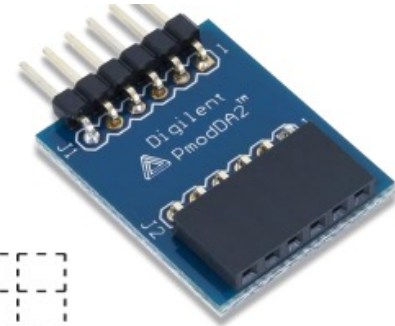
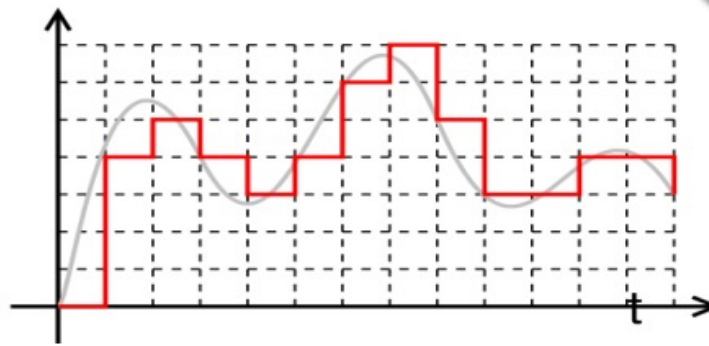
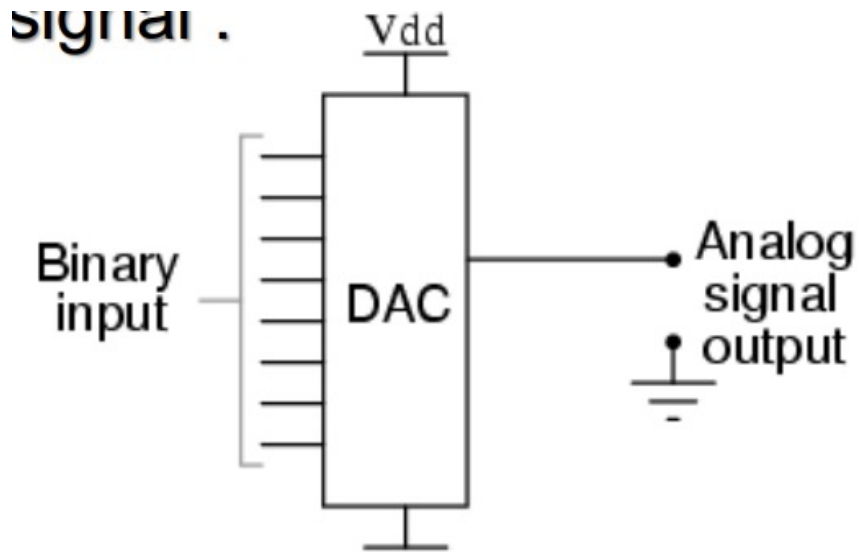
SPI Example: ADC

- The SYNC signal is pin je[1], the two output data signals ADCD0 and ADCD1 are pins je[2] and je[3] and the SPI bus clock signal SCLK is pin je[4]. Pins 5 and 6 are DC ground and power ($V_{DD} = +3.3 \text{ V DC}$).



SPI Example: DAC

- Provides an analog output signal for the discrete sampled data of an application as a step-wise but continuous analog output signal .



SPI Example: DAC

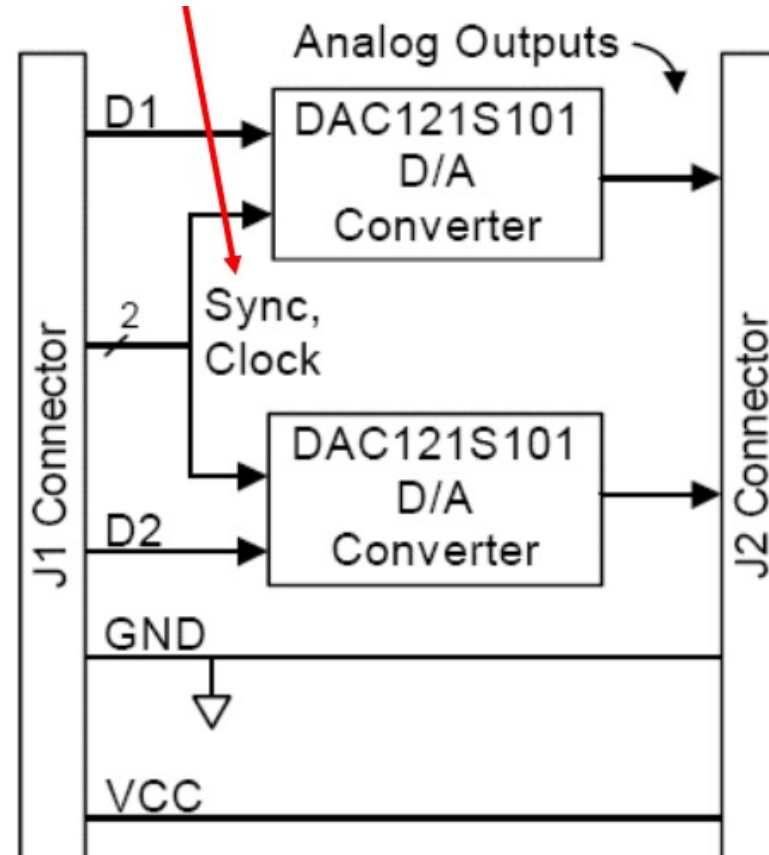
- The analog output voltage resolution (or step size) ΔV for a DAC that inputs an unsigned n-bit binary values is determined by:

$$\Delta V = \frac{V_{\text{REF}}}{2^n}$$

- V_{REF} is the precision DC voltage used as a reference. The maximum rate at which the DAC can accept binary data and update its analog voltage output is its throughput rate R_{DAC} in samples per second.

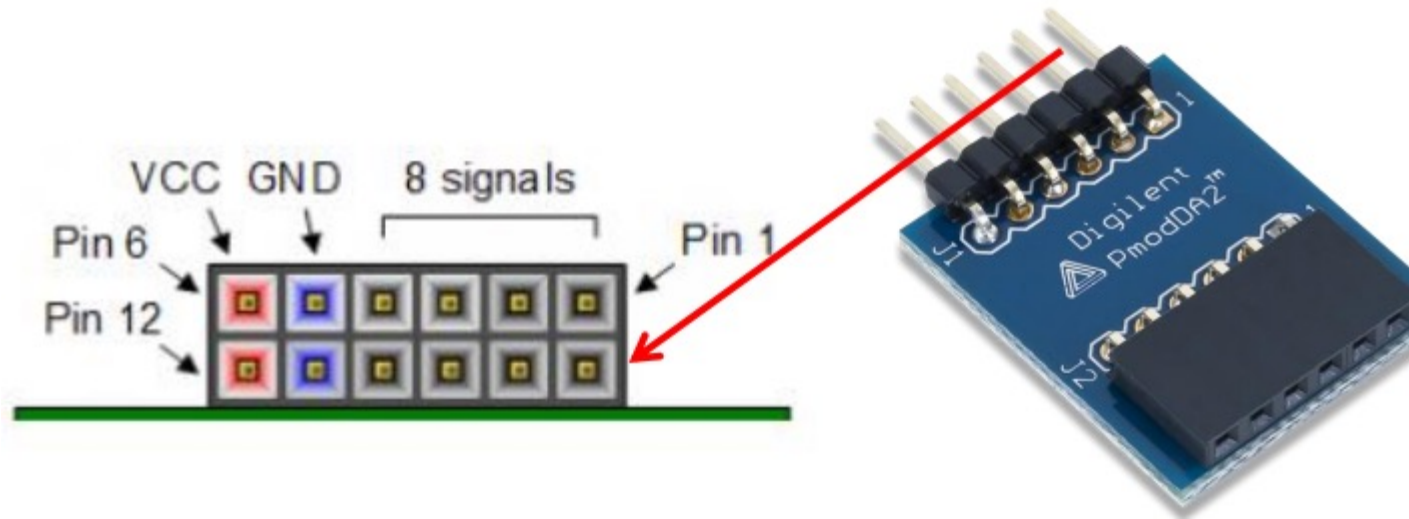
SPI Example: ADC

- The PmodDA2 hardware module features two Texas Instruments (www.ti.com) DAC121S101 12-bit DACs and utilizes the SPI protocol with a 16-bit data packet (12 bits of data and 4 bits of control). SYNC is the slave select signal SS.



SPI Example: ADC

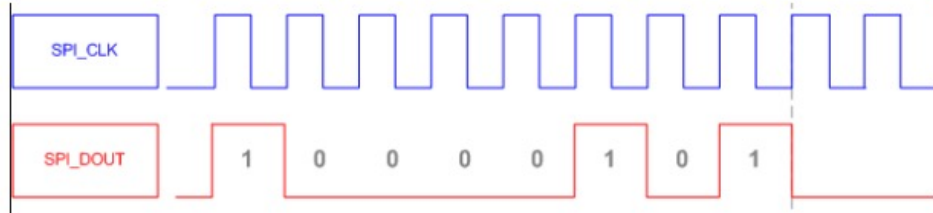
- The PmodDA2 DAC is connected to the bottom 6 pins of the 12-pin peripheral hardware module connector (je) of the Zybo Board.



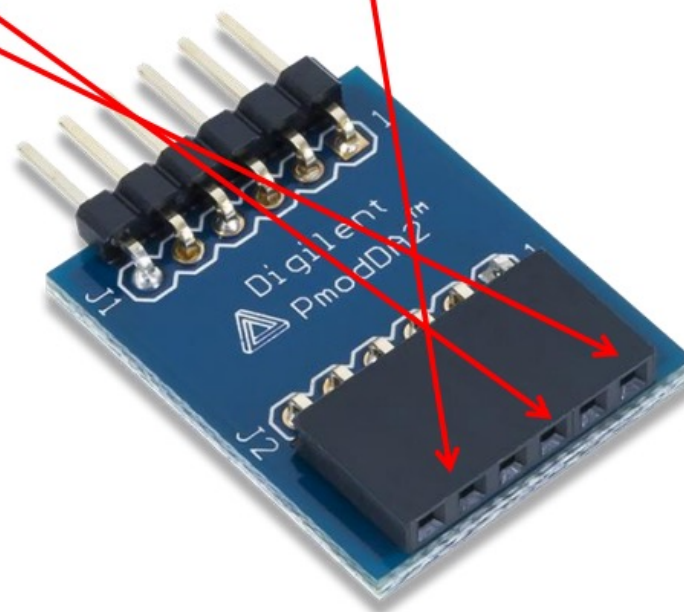
- The SYNC (or SS) signal is pin je[7], the two input data signals DACD0 and DACD1 are pins je[8] and je[9] and the SPI bus clock signal SCLK is pin je[10]. Pins je[11] and je[12] are DC ground and power ($V_{DD} = +3.3 \text{ V DC}$).

SPI Example: ADC

- The analog output voltage for the 12-bit DAC is determined by the equation below with $n = 12$, $D[11:0]$ ranging from 0 to 4095 and $V_{REF} = 3.3 \text{ V}$. The two unipolar analog output voltages A and B appear on a 6-pin header at pins 1 and 3 and ground at pin 5.



$$V_{OUT} = \frac{D[n-1:0] \times V_{REF}}{2^n} \text{ V}$$



End

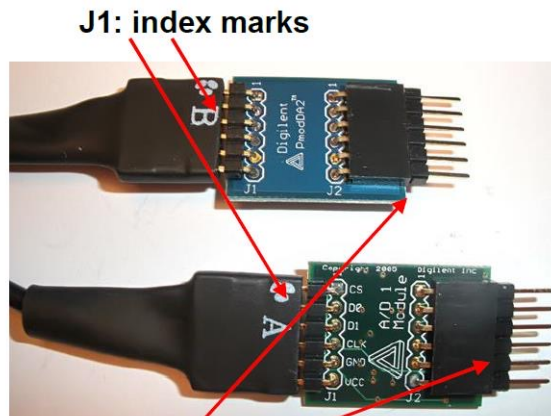
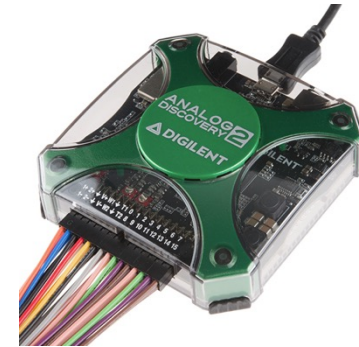
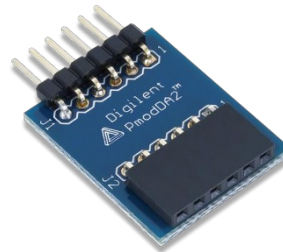
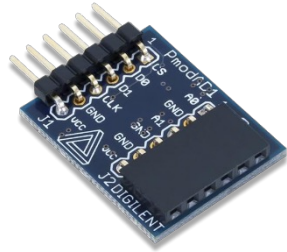
- Full Content Refer: Zynq SPI Peripherals ADC and DAC.pdf

Lab 5: SPI ADC

Laboratory Assignment 5		✓	+	⋮
⋮	📎 LAB 5 PmodAD1 Vivado SDK Project.docx	✓		⋮
⋮	📎 Zynq SPI Peripherals ADC & DAC.pdf	✓		⋮
⋮	📎 Lab5Files.zip	✓		⋮
⋮	📎 diligent.waveforms_v3.17.1_64bit.exe	✓		⋮
⋮	📎 xilinx.com_user_AD1Pmod_1.0.zip	✓		⋮
⋮	📎 ZynqPmodVivado-Library.zip	✓		⋮
⋮	📄 Lab 5 report Oct 21 100 pts	✓		⋮

Hardware Components and Setup

- ADC
- DAC (Lab6)
- Diligent Discovery Board



J2: 6 pin headers input or output connectors

The Digilent Zybo Board Pmod connectors

- Pmod port JA is the 12-bit Xilinx analog-to-digital converter (XADC) and not used in this course.
- Pmod port JF is connected to the MIO bus of the processor system (PS) of the Zynq system-on-chip (SoC) via 200 Ohm series resistors.

Pmod JA (XADC)	Pmod JB (Hi-Speed)	Pmod JC (Hi-Speed)	Pmod JD (Hi-Speed)	Pmod JE (Std.)	Pmod JF (MIO)
JA1: N15	JB1: T20	JC1: V15	JD1: T14	JE1: V12	JF1: MIO-13
JA2: L14	JB2: U20	JC2: W15	JD2: T15	JE2: W16	JF2: MIO-10
JA3: K16	JB3: V20	JC3: T11	JD3: P14	JE3: J15	JF3: MIO-11
JA4: K14	JB4: W20	JC4: T10	JD4: R14	JE4: H15	JF4: MIO-12
JA7: N16	JB7: Y18	JC7: W14	JD7: U14	JE7: V13	JF7: MIO-0
JA8: L15	JB8: Y19	JC8: Y14	JD8: U15	JE8: U17	JF8: MIO-9
JA9: J16	JB9: W18	JC9: T12	JD9: V17	JE9: T17	JF9: MIO-14
JA10: J14	JB10: W19	JC10: U12	JD10: V18	JE10: Y17	JF10: MIO-15

The Digilent Zybo Board Pmod connectors

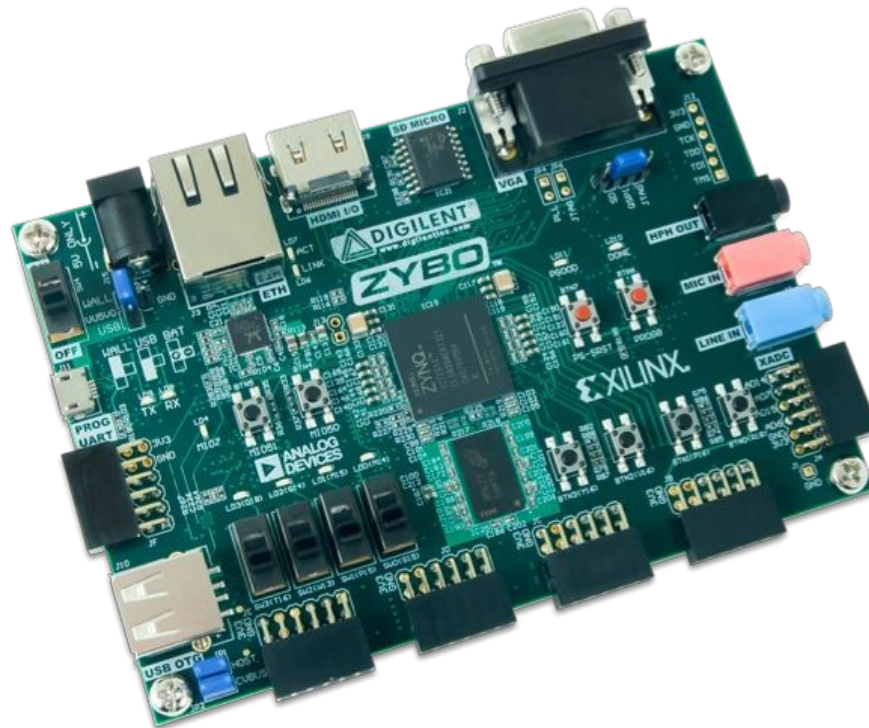
- Pmod port JE is connected to the programmable logic (PL) of the Zynq SoC device via 200 Ohm series resistors.
- Pmods JB, JC and JD are the high-speed connectors and have their data signals routed as impedance matched differential pairs for maximum switching speeds.

Pmod JA (XADC)	Pmod JB (Hi-Speed)	Pmod JC (Hi-Speed)	Pmod JD (Hi-Speed)	Pmod JE (Std.)	Pmod JF (MIO)
JA1: N15	JB1: T20	JC1: V15	JD1: T14	JE1: V12	JF1: MIO-13
JA2: L14	JB2: U20	JC2: W15	JD2: T15	JE2: W16	JF2: MIO-10
JA3: K16	JB3: V20	JC3: T11	JD3: P14	JE3: J15	JF3: MIO-11
JA4: K14	JB4: W20	JC4: T10	JD4: R14	JE4: H15	JF4: MIO-12
JA7: N16	JB7: Y18	JC7: W14	JD7: U14	JE7: V13	JF7: MIO-0
JA8: L15	JB8: Y19	JC8: Y14	JD8: U15	JE8: U17	JF8: MIO-9
JA9: J16	JB9: W18	JC9: T12	JD9: V17	JE9: T17	JF9: MIO-14
JA10: J14	JB10: W19	JC10: U12	JD10: V18	JE10: Y17	JF10: MIO-15

More details refer: [Digilent Zybo Pmod Connector.pdf](#)

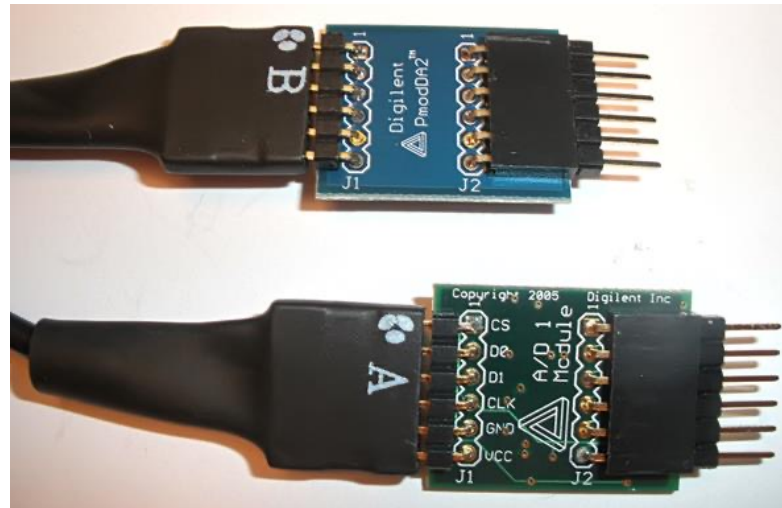
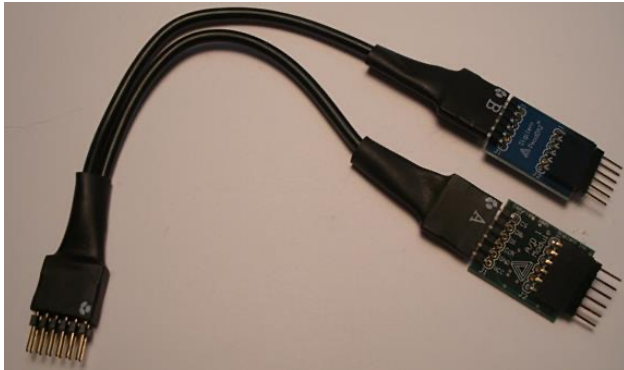
PmodAD1

- The Digilent PmodAD1 and PmodDA2 provide this external functionality and both 6-pin peripheral devices can be installed on the 12-pin Pmod jack on the lower left side of the Zybo Board.



PmodAD1

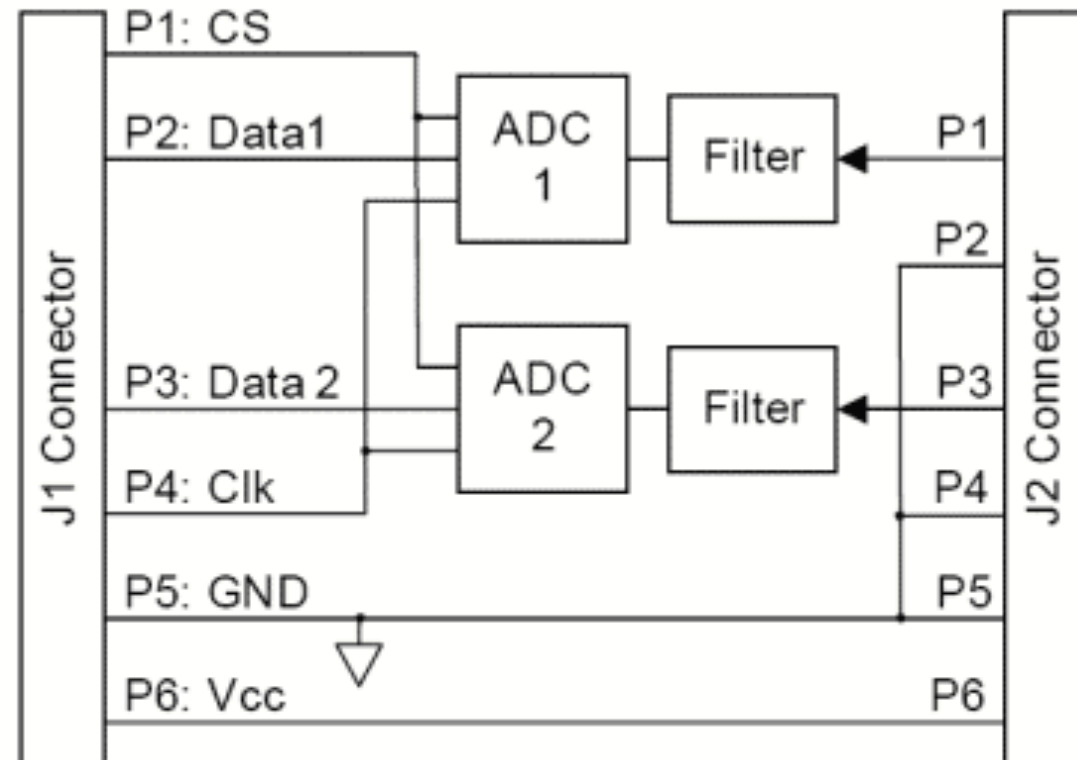
- The PmodAD1 ADC is connected to the cable 6 pin connector A with the index mark positioned on pin 1 of J1 the SPI interface, as shown in the Pictures.



!!! Improper connection can damage the Zybo Board and the Pmod peripherals

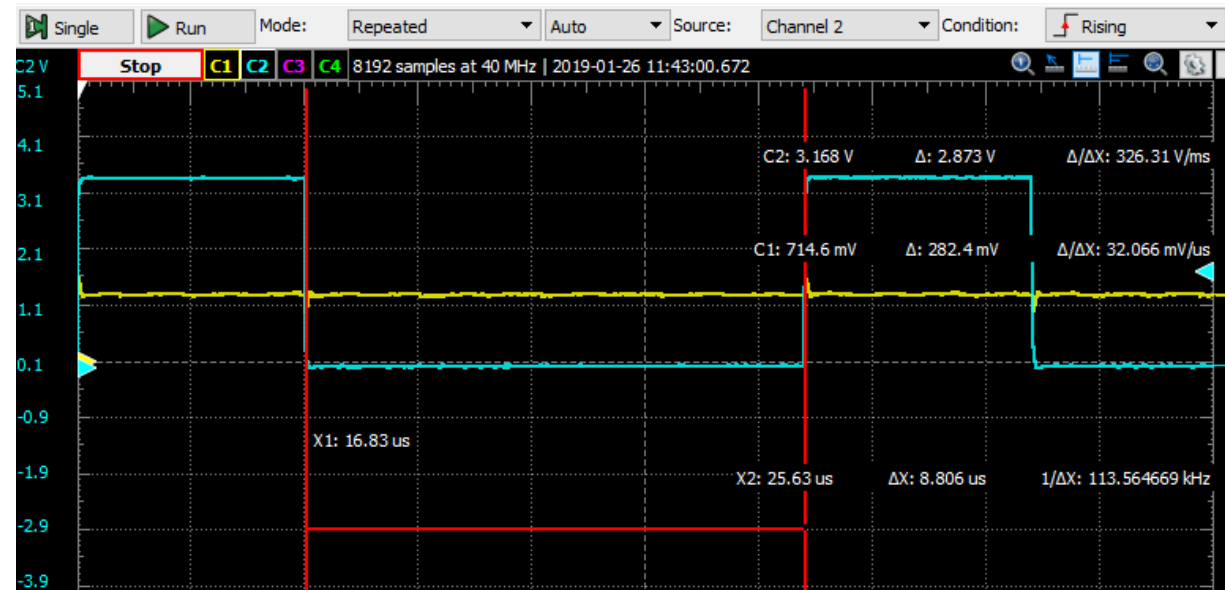
PmodAD1

- The digital output D for an analog input voltage V_{IN} is given by the equation below. With $V_{REF} = 0V$, $V_{FS} = 3.3V$, $G=1$ and $n=12$ the PmodAD1 output data D ranges from 0 to 4095 for an analog signal input V_{IN} of approximately 0 to 3.30 V



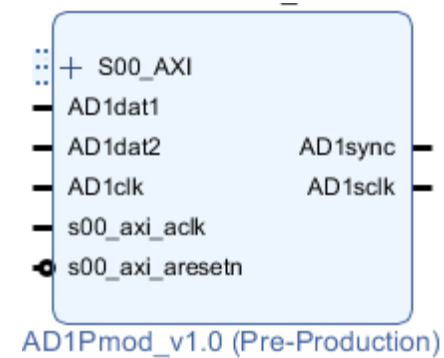
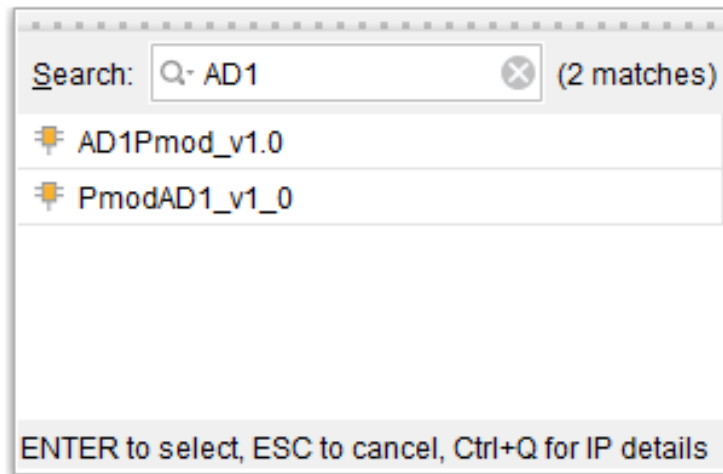
Digilent Discovery 2 Board and the Waveforms

- Input a DC voltage (0V, 1V, 2V, 3V and 4V) from the Digilent Discovery 2 Waveform Generator to channel 1 of the ADC.



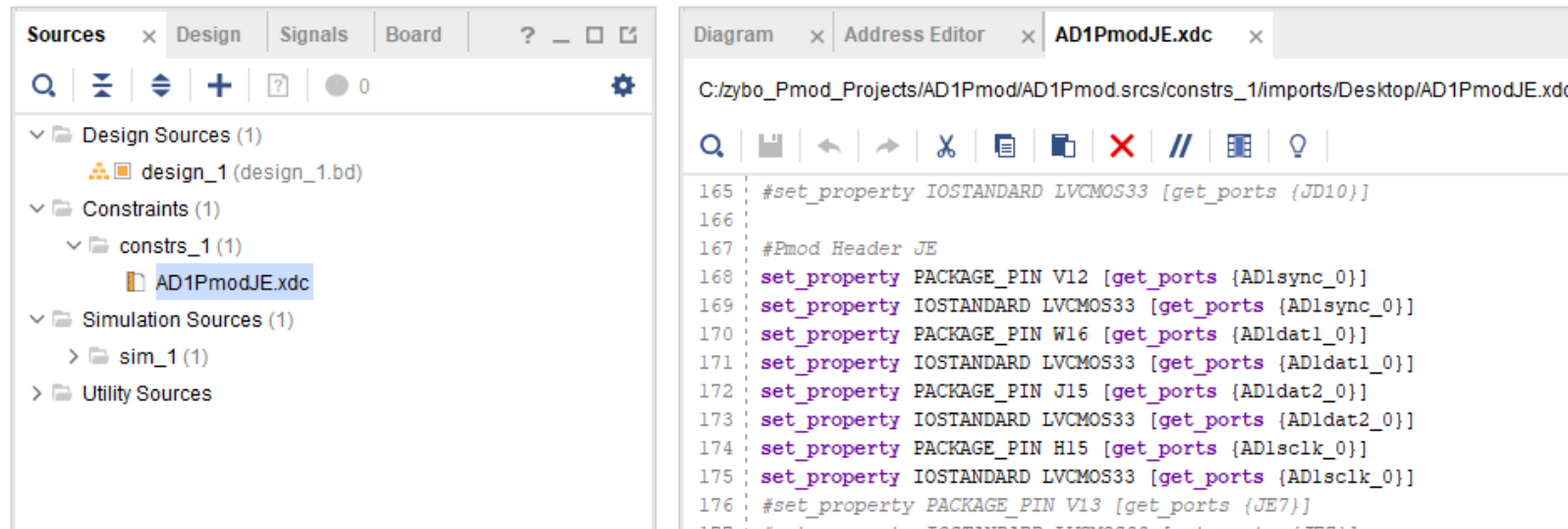
Vivado project Configuration

- Add IP for AD1Pmod



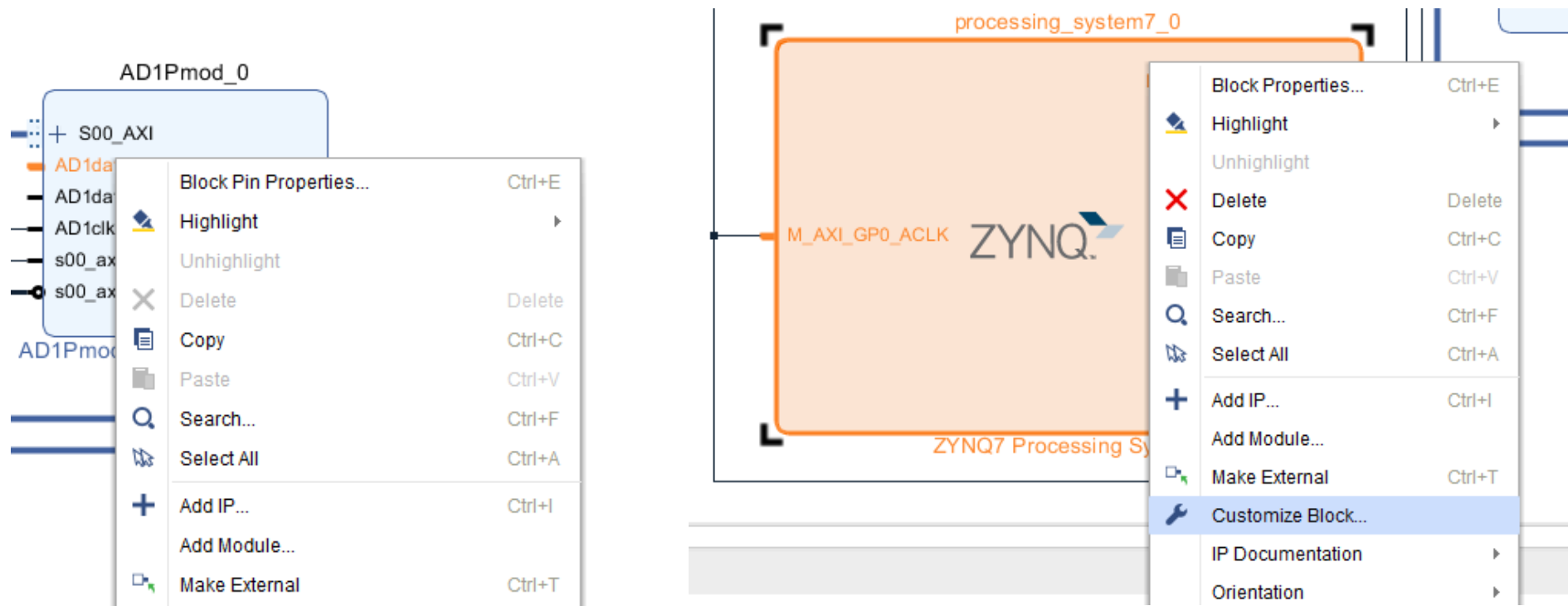
Vivado project Configuration

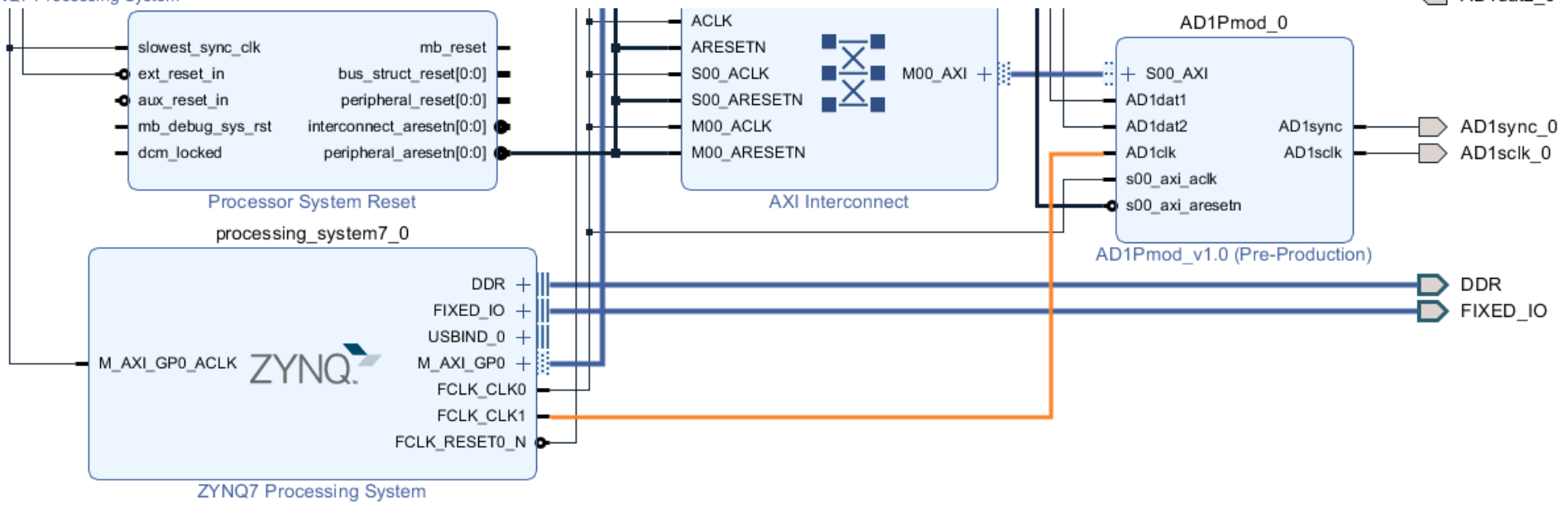
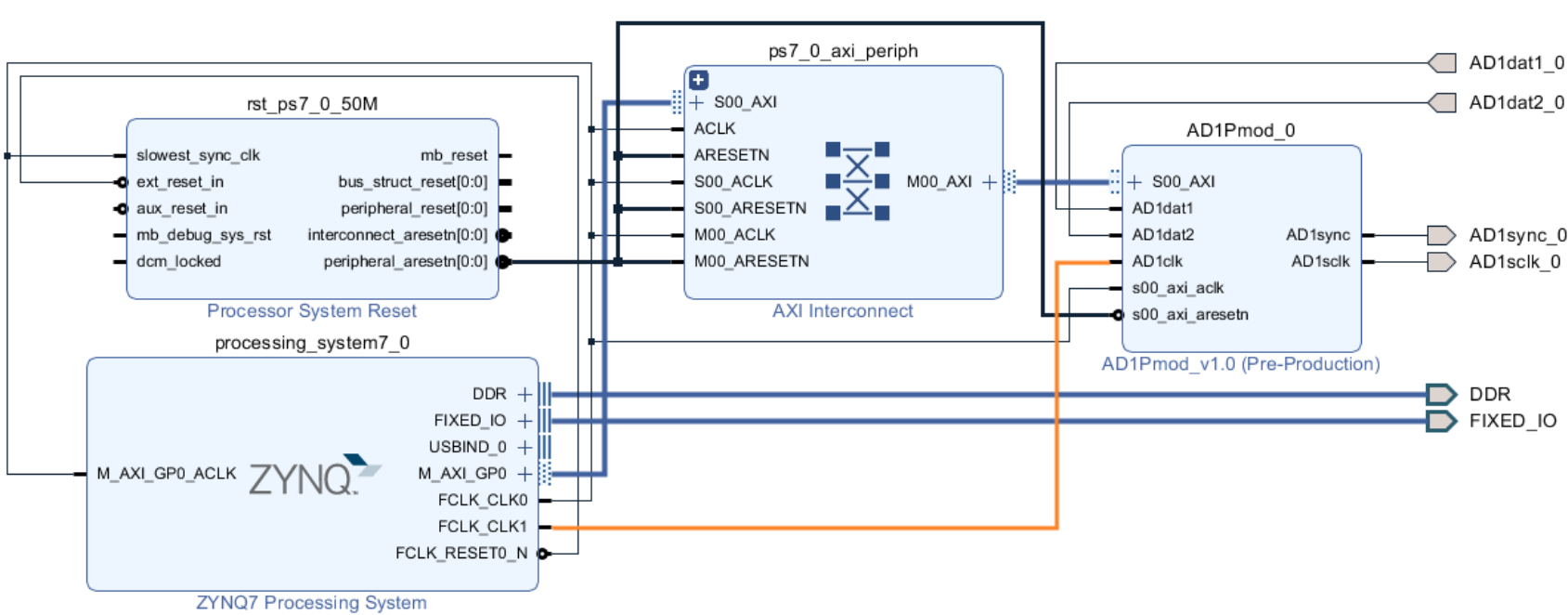
- Add Constraints:
 - The Pmod1 AD named connections to the top row of port JE (left most, bottom).



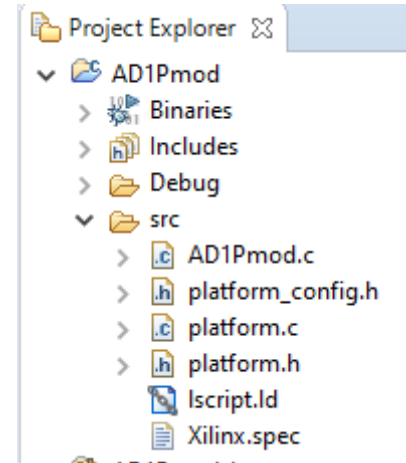
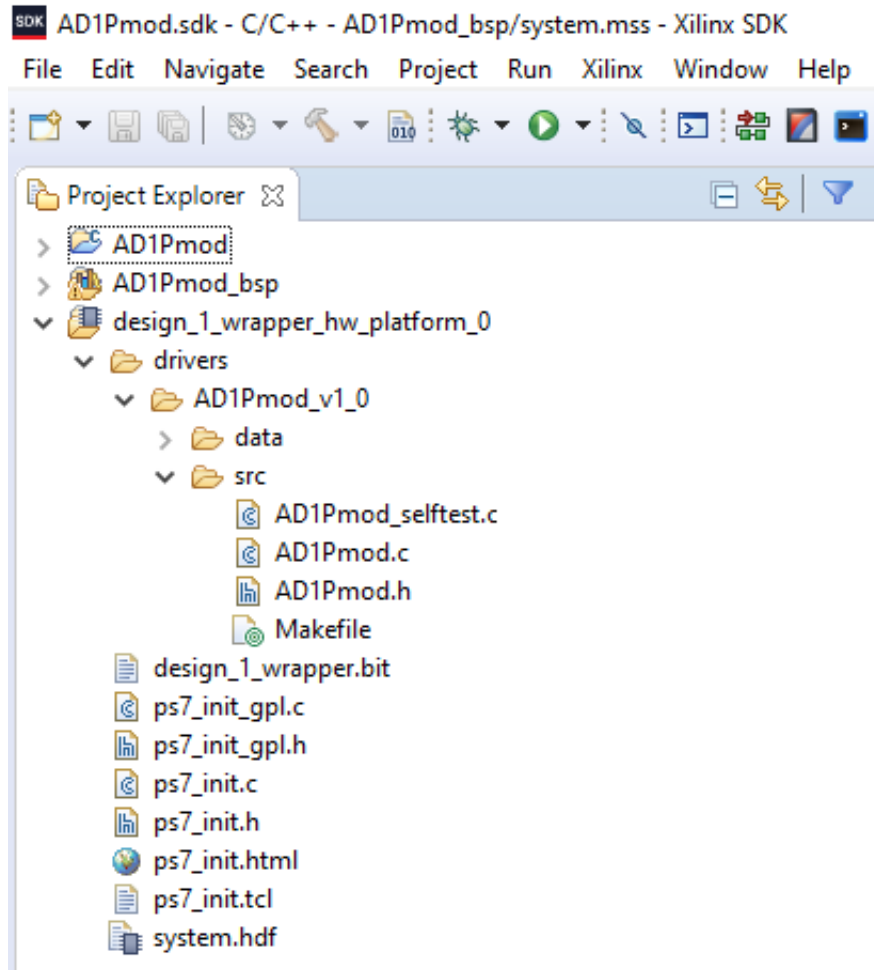
Vivado project Configuration

- Make External for 4 unconnected pins
 - AD1dat1, AD1dat2, AD1Sync, AD1CLK
- External Clock





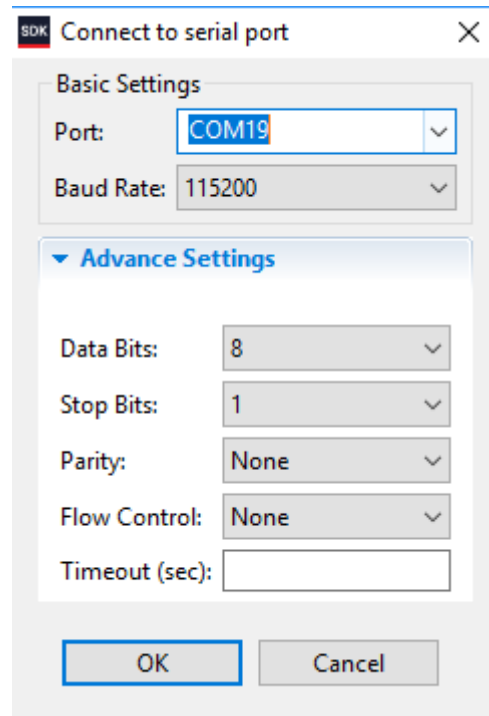
SDK Application



Download *AD1Pmod.c*
from Canvas

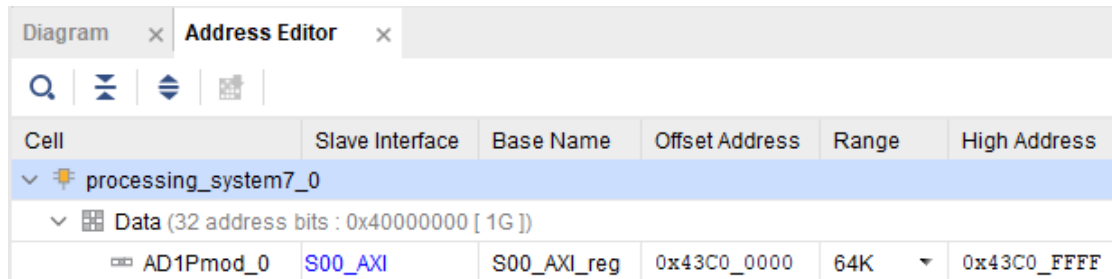
SDK Application

- The Port (may be) automatically identified. This is the same USB port used for programming the Zybo and providing power.



SDK Application

- The AD1Pmod base addresses are in the Address Editor in Vivado and are 32 bit registers (4 bytes).



The screenshot shows the Vivado Address Editor window. The title bar includes tabs for 'Diagram' and 'Address Editor'. Below the title bar is a toolbar with icons for search, zoom, and other functions. The main area is a table with columns: Cell, Slave Interface, Base Name, Offset Address, Range, and High Address. The table is expanded to show the 'processing_system7_0' cell, which contains a 'Data (32 address bits : 0x40000000 [1G])' entry. This entry is further expanded to show the 'AD1Pmod_0' register, which is a 32-bit register (4 bytes) located at offset 0x43C0_0000, with a range of 64K, and a high address of 0x43C0_FFFF.

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
processing_system7_0					
Data (32 address bits : 0x40000000 [1G])					
AD1Pmod_0	S00_AXI	S00_AXI_reg	0x43C0_0000	64K	0x43C0_FFFF

SDK Application

- Codes for reading data

Details refer: SPI ADC and DAC Peripherals.pdf

```
//AD1Pmod.c PmodAD1 ADC example ECE3622 c2019 Dennis Silage

#include "xparameters.h"
#include "xil_io.h"
#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include <sleep.h>

//AD1Pmod from Address Editor in Vivado
#define AD1acq 0x43C00000 //AD1 acquisition
#define AD1dav 0x43C00004 //AD1 data available
#define AD1dat1 0x43C00008 //AD1 channel 1 data
#define AD1dat2 0x43C0000C //AD1 channel 2 data

int main(void)
{
    int adcdav; //ADC data available
    int adcdat1; //ADC channel 1 data
    int adcdat2; //ADC channel 2 data

    xil_printf("\n\rStarting AD1 Pmod demo test...\n");
    Xil_Out32(AD1acq,0);

    while (1)
    {
        Xil_Out32(AD1acq,0); //ADC stop acquire
        adcdav=Xil_In32(AD1dav); //ADC available?
        while(adcdav==1)
            adcdav=Xil_In32(AD1dav);

        while(1)
        {
            Xil_Out32(AD1acq,1); //ADC acquire
            while(adcdav==0)
                adcdav=Xil_In32(AD1dav); //ADC data?
            Xil_Out32(AD1acq,0); //ADC stop acquire
            adcdat1=Xil_In32(AD1dat1);
            adcdat2=Xil_In32(AD1dat2);
            while(adcdav==1)
                adcdav=Xil_In32(AD1dav); //ADC reset?
            xil_printf("%d Ch1 %d, Ch2 %d\n", adcdat1, adcdat2);
            sleep(1);
        }
    }
}
```

Lab 5 Requirement

- Step 1:
- Execute the example PmodAD1 project in Vivado SDK
- The PmodAD1 is connected to JE on the Zybo Board. Setup the Discovery 2 and input a DC voltage (0V, 1V, 2V, 3V and 4V) from the Digilent Discovery 2 Waveform Generator to channel 1 of the ADC.
- What is the exact saturation voltage for the ADC?

Lab 5 Requirement

- Step 2: Modify the PmodAD1 project.
 - a) In the Vivado hardware block diagram, add the LEDs as a GPIO, and then in SDK, map the 12-bit ADC output to the 4-bit LEDs. For example, if the ADC output is between 0-???, then the LED pattern should be 0000. Conversely, if the ADC output is between ???-4096, then the LED pattern should be 1111. Determine and map all the values in between the 12-bit range 0-4096, so that the LEDs will illuminate based on the ADC values. (Hint: divide the range of the range of the 12-bit ADC by the number of possible outcomes of the 4-bit LEDs)
 - b) Input a unipolar ramp waveform from the Digilent Discovery 2 Waveform Generator to channel 1 of the ADC. The ramp waveform should periodically rise from 0v – saturation voltage determined in part 1 above. Make the period of the ramp waveform slow enough to be sampled by the ADC at least 16 times during each period. This will force the LEDs to count up with each ADC sample.

Lab 5 Requirement

```
• while(1)
• {
•     Xil_Out32(AD1acq,1);          //ADC acquire
•     while(adcdav==0)
•         adcdav=Xil_In32(AD1dav); //ADC data?
•     Xil_Out32(AD1acq,0);          //ADC stop acquire
•     adcdat1=Xil_In32(AD1dat1);
•     adcdat2=Xil_In32(AD1dat2);
•     while(adcdav==1)
•         adcdav=Xil_In32(AD1dav); //ADC reset?
•     xil_printf("%d Ch1 %d, Ch2 \n", adcdat1, adcdat2);
•     sleep(1);
•
•     //12bit to 4bit Mapping Code Goes Here
•
•     If(adcdat1<255) led=0b0000;
•     Else If(adcdat1<511) led=0b0001;
•
•     .
•
•     .
•
•     Else led=0b1111;
•
•     Xgpio_discretewrite(&LEDinst, 1, led);
•
• }
```

Details refer: LAB 5 PmodAD1 Vivado SDK Project.docx